

Placement Portal

Modern Application Development II — Project Report

A Flask (REST API) + Vue.js single-page application for managing campus recruitment

1. Student Details

Name: Archita Jain

Roll Number: 23f3001889

Email: 23f3001889@ds.study.iitm.ac.in

2. Project Details

Project Title: Placement Portal

Problem Statement:

To design and build a multi-role web application for managing campus recruitment. The platform provides a centralised system for posting placement drives, managing applications, and tracking candidate progress through the hiring pipeline, for three distinct user roles: Admin, Company, and Student.

Approach:

The MAD-1 version of this project was built as a server-rendered Flask application using Flask-Login and Jinja2 templates. For MAD-2, the backend was re-architected into a pure JSON REST API using the Flask Application Factory pattern with Blueprints, and session-based auth was replaced with flask-jwt-extended for stateless, token-based authentication. The frontend was rebuilt as a Vue.js single-page application (served by Flask, with client-side routing via Vue Router) that consumes these APIs, with flask-cors enabling cross-origin requests during development. Long-running work such as interview reminders, CSV exports, and monthly PDF report generation was moved off the request/response cycle into Celery background tasks backed by Redis, with Celery Beat handling scheduled jobs and flask-caching used to speed up frequently read endpoints.

3. AI/LLM Declaration

Claude (Anthropic) and ChatGPT were used as development assistants, primarily for debugging Vue 3 component logic, resolving Celery/Redis background task configurations, and generating repetitive Bootstrap boilerplate for the CRUD views. All core design, database schemas, authentication flows, and final code integrations were done manually. Estimated usage: ~10–15%, limited strictly to code suggestions, troubleshooting, and documentation formatting.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework, structured with the Application Factory pattern and Blueprints
Flask-RESTful	Resource-based REST endpoints (e.g. /api/stats, /api/students, /api/drives)
Flask-SQLAlchemy / SQLAlchemy	ORM for the SQLite database and model relationships
flask-jwt-extended	Stateless JWT authentication, replacing Flask-Login from MAD-1
flask-cors	Enables the Vue frontend to call the Flask API across origins
Vue.js (+ Vue Router)	Single-page application frontend; role-based layouts and views for Admin, Company and Student
Celery + Redis	Asynchronous background tasks — interview reminders, bulk exports, scheduled monthly reports
flask-caching	Server-side caching of frequently accessed endpoints

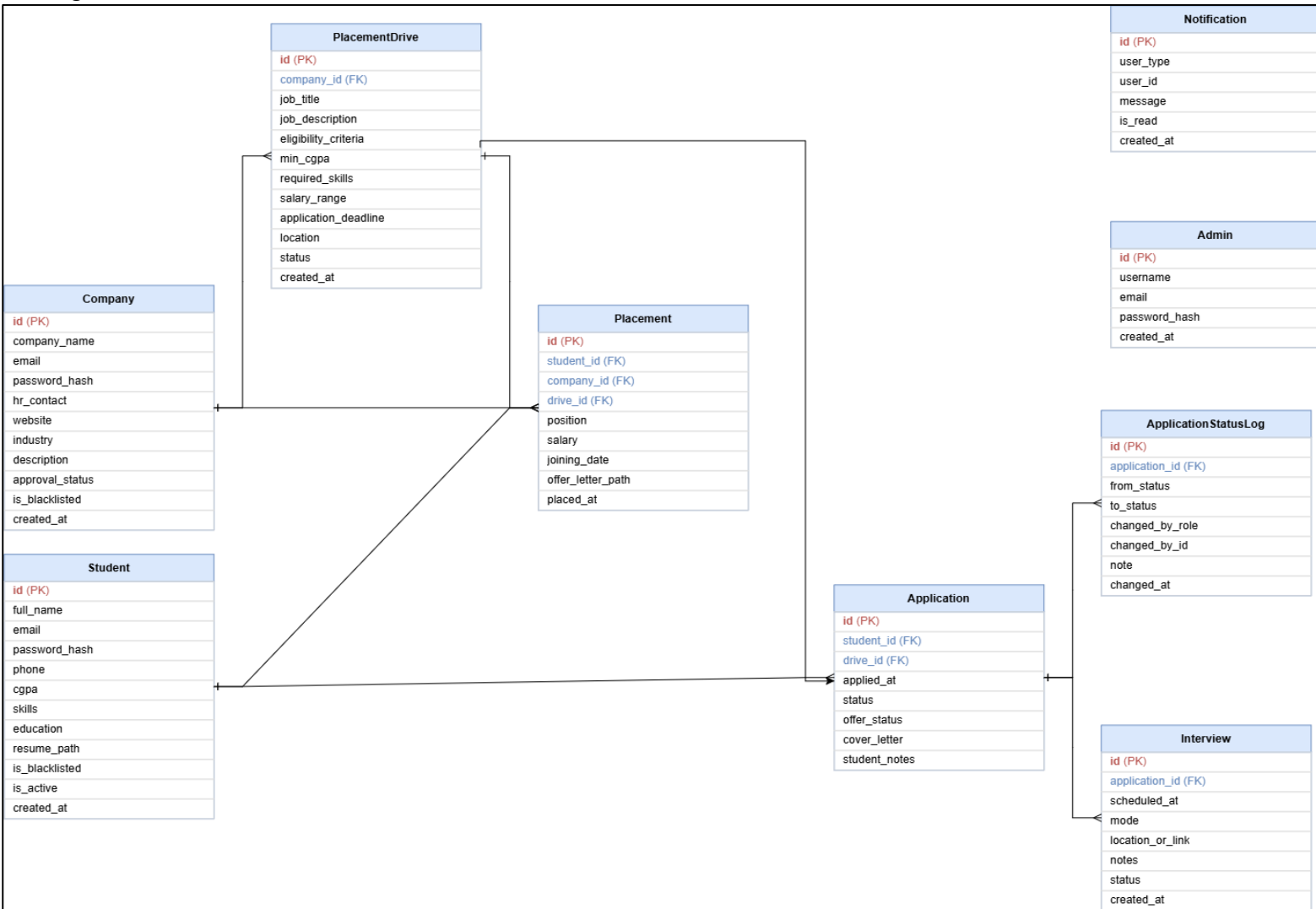
Technology / Library	Purpose
fpdf2	Generates PDF placement/admin/company reports sent out by Celery tasks
Werkzeug	Password hashing and secure file upload utilities
SQLite	Lightweight relational database (instance/placement_portal.db)

5. Database Schema / ER Diagram

Core tables and relationships:

Table	Key Columns	Relationships
Admin	id, username, email, password_hash	—
Company	id, company_name, email, approval_status, is_blacklisted	1 → Many PlacementDrive
Student	id, full_name, email, cgpa, skills, resume_path, is_blacklisted	1 → Many Application
PlacementDrive	id, company_id (FK), job_title, min_cgpa, application_deadline, status	Belongs to Company; 1 → Many Application
Application	id, student_id (FK), drive_id (FK), status, offer_status	Belongs to Student & Drive; 1 → Many Interview, ApplicationStatusLog
ApplicationStatusLog	id, application_id (FK), from_status, to_status, changed_by_role	Audit trail of every Application status change
Interview	id, application_id (FK), scheduled_at, mode, status	Belongs to Application
Placement	id, student_id, company_id, drive_id (FKs), position, salary, offer_letter_path	Links Student, Company and Drive on a final placement
Notification	id, user_type, user_id, message, is_read	Indexed on (user_type, user_id)

ER Diagram:



6. API Resource Endpoints

The API is organised into four Blueprints (auth, admin, company, student) plus a small Flask-RESTful resource layer.

Representative endpoints:

Endpoint	Method	Description
/api/auth/login	POST	Authenticate a user (any role) and issue a JWT
/api/auth/register/student	POST	Register a new student account
/api/auth/register/company	POST	Register a new company account (pending admin approval)
/api/student/drives	GET	List placement drives visible/eligible to the logged-in student
/api/student/applications	POST, GET	Apply to a drive; list the student's own applications
/api/company/drives	GET, POST	List or create placement drives for the logged-in company
/api/company/applications/<id>/status	PUT	Update an applicant's status (Shortlisted, Selected, Rejected, etc.)
/api/admin/companies/<id>/approve	PUT	Approve a pending company registration

Endpoint	Method	Description
/api/admin/trigger-monthly-report	POST	Manually trigger the Celery monthly PDF report task
/api/stats	GET	Aggregate counts — students, companies, drives, applications
/api/students	GET, POST	List all students; create a new student (admin)
/api/students/<id>	GET, PUT, DELETE	Retrieve, update, or delete a specific student

All endpoints accept and return JSON and are protected by JWT where applicable. The complete OpenAPI (YAML) definition is submitted as a separate api.yaml file alongside this report.

7. Architecture and Features

Architecture Overview:

- app.py — Application Factory (create_app()); initialises SQLAlchemy, JWT, CORS, cache and Celery, registers all Blueprints, and defines a catch-all route that serves the Vue SPA (templates/index.html)
- routes/auth.py, admin.py, company.py, student.py — JSON API controllers per role, guarded by decorators.py (admin_required, company_required, student_required)
- routes/api.py — Flask-RESTful resources for simple CRUD (/api/stats, /api/students, /api/drives)
- models.py — SQLAlchemy models and relationships; constants.py centralises status enums (ApplicationStatus, DriveStatus, InterviewStatus, OfferStatus)
- tasks.py / reports.py / celery_worker.py — Celery tasks for interview reminders, exports, and monthly report emails; fpdf2-based PDF layouts
- static/js/components/ — Vue components split by role (admin/, company/, student/) plus shared layouts and a router.js for client-side navigation
- static/uploads/resumes/, static/reports/ — file storage for uploaded resumes and generated PDF reports

Implemented Features:

- JWT-based authentication for Admin, Company, and Student roles, replacing MAD-1's session-based login
- Admin panel: approve/reject/blacklist companies and students, manage drives, view aggregate stats and search across entities
- Company panel: post and manage placement drives, review applicants, bulk/individual status updates, schedule interviews
- Student panel: browse and apply to drives, track application status and history, manage profile and resume, view notifications and offers
- Asynchronous CSV export of applications/drives with task-status polling endpoints
- Scheduled and on-demand PDF report generation (admin and company) emailed automatically via Celery Beat
- Server-side response-time header and caching (flask-caching) for frequently accessed read endpoints
- Full audit trail of application status transitions via ApplicationStatusLog

8. Video Presentation

Drive Link: <https://drive.google.com/file/d/1EjtwYBWLIXNE167C53v7HddTHoV5pV2j/view?usp=sharing>